1️⃣ From “hard-coded logic” to “decision making”

In the first two programs, the filtering logic is **hard-wired** inside the loop.

For example, the program literally checks:

```js
if (givenArray[i] < maxNumber) { ... }
```

or

```js
if (givenArray[i] % num == 0) { ... }
```

The loop does **two jobs at once**:

1. Go through every element
2. Decide whether the element should be kept

This works, but it is **not flexible**.
If you want a new rule (e.g. “greater than 70”), you must write a new function.

In Program 2, this logic is made clearer by pulling the condition into a variable:

```js
const isWhatIWant = (givenArray[i] < maxNumber);
```

programFilter02

This is the first hint that **filtering is really about evaluating a true/false decision**.

2 Turning the decision into a function

In Program 3, the rule is separated from the loop:

```js
function checkLessThan50(givenNumber) {
    return (givenNumber < 50);
}
```

Now the loop does this:

```js
const isWhatIWant = checkWithThisFunction(givenArray[i]);
```

programFilter03

This means:

“For every number in the array, ask another function whether this number should be kept.”

This other function is what we call a **callback function**.

The loop does not know the rule — it only knows:

“Call this function and expect a true or false back.”

3 Generalising into myFilter

In Program 4, both “less than 50” and “divisible by 3” are handled by the **same** function:

```js
function myFilter(givenArray, checkWithThisFunction) { ... }
```

programFilter04

The algorithm is now clear:

For every element:

1. Send the element into the callback
2. Get back true or false
3. If true, push it into the result array

So conceptually:

```sql
result = []
for each element in array:
    if callback(element) is true:
        add element to result
return result
```

This is exactly what **filter** means.

4 Callback functions can be created on the fly

In Program 5, instead of writing named functions like checkLessThan50, the callback is written directly:

```js
myFilter(numberArr, givenNumber => givenNumber < 50);
```

programFilter05

This works because JavaScript allows functions to be passed like values.
The filter algorithm does not care **where the function came from** — only that it returns true or false.

5 The built-in Array.filter() is the same idea

Finally, Program 6 shows:

```js
numberArr.filter(givenNumber => givenNumber < 50);
```

programFilter06

This means:

"JavaScript now provides the myFilter function for you — built into every array."

Internally, Array.filter() is doing almost exactly what your myFilter function did:

```sql
create empty result array
for each element in the array:
    call the callback function with that element
    if callback returns true:
        put the element into result
return result
```

🎯 Final Concept Students Should Understand

Array.filter() **does not know what you are looking for**.

It only knows how to:

- Walk through the array
- Call your callback function on every element
- Keep the elements for which your callback returns true

So when you write:

```
js

numbers.filter(n => n % 3 == 0)
```

You are not telling JavaScript **how to loop** —
you are only telling it **how to decide**.

That separation of:

“how to loop” vs **“how to decide”**

is what makes higher-order functions like filter() so powerful.

~ END ~